

Análisis de Sistemas

Materia:
Análisis y Metodología de
Sistemas

Docente contenidista: QUIRÓZ, Gamaliel

Revisión: Coordinación

Contenido

Introducción	04
Diagrama Entidad-Relación (DER)	04
Errores comunes en el balanceo	10
Programación orientada a objetos	12
Polimorfismo	16
Diferencias con lenguaje de programación java	17
Bibliografía	19

Clase 5



¡Te damos la bienvenida a la materia
Análisis y Metodología de Sistemas!

En esta clase vamos a ver los siguientes temas:

- Diagrama Entidad Relación.
- Uso de diagramas.
- Balanceo entre diagramas.
- Programación Orientada a Objetos en Python.

Introducción

En la clase anterior trabajamos con herramientas clave para el análisis estructurado de sistemas: el diccionario de datos, las interfaces de usuario y conceptos de usabilidad y experiencia del usuario (UX/UI). Aprendimos cómo describir los datos de forma detallada, clara y coherente, y cómo una buena interfaz puede impactar positivamente en la interacción del usuario con el sistema. También revisamos la importancia de representar la lógica del sistema a través de estructuras claras y bien definidas.

En esta clase, avanzamos en la representación lógica de la información enfocándonos en el modelado de datos. Vamos a estudiar el Diagrama Entidad-Relación (DER), una herramienta fundamental para comprender la estructura de los datos en un sistema, y luego veremos cómo se realiza el balanceo entre modelos, un paso clave para asegurar la coherencia entre las distintas vistas del sistema.

Diagrama Entidad-Relación (DER)

El Diagrama Entidad-Relación (DER) es una herramienta de modelado que permite representar gráficamente la estructura lógica de los datos que utiliza un sistema de información. Fue propuesto por Peter Chen en 1976 y se convirtió en un estándar de hecho para el análisis y diseño de bases de datos relacionales.

¿Para qué sirve un DER?

- Representa las entidades relevantes del sistema.
- Muestra cómo se relacionan esas entidades entre sí.
- Describe las propiedades o atributos de cada entidad y relación.
- Sirve como base para el diseño físico de una base de datos relacional.
- Ayuda a identificar qué datos se necesitan y cómo están organizados.
- Facilita el diálogo entre analistas, desarrolladores y usuarios.

El DER es útil para visualizar cómo están organizados los datos y cómo se conectan entre sí. Representa de manera lógica la **estructura de datos** que el sistema debe manejar.

Las relaciones que se establecen entre entidades reflejan la lógica del negocio o del sistema, mostrando cómo interactúan los distintos componentes de la información.

Componentes principales del DER

Los tres componentes básicos del Diagrama Entidad-Relación son:

- **Entidades.**
- **Relaciones.**
- **Atributos.**

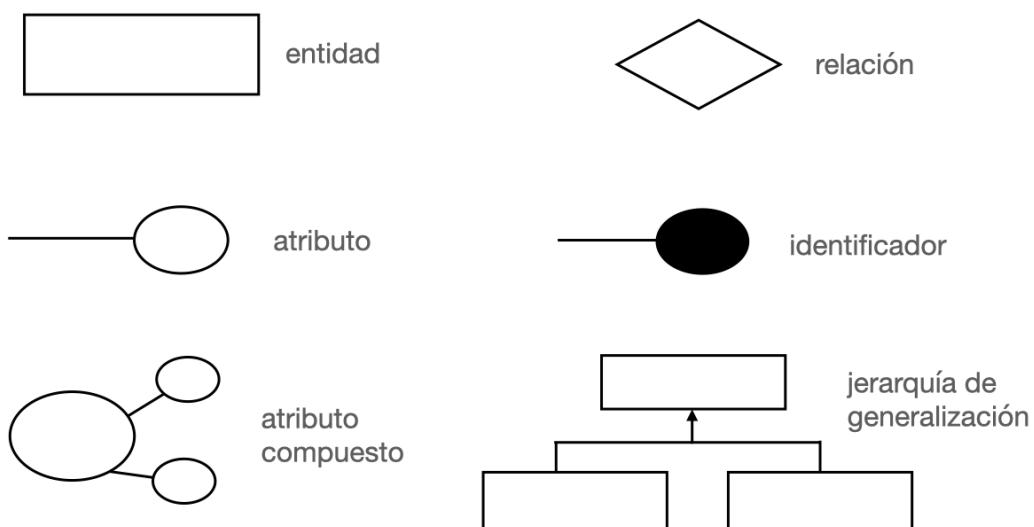


Imagen: Elementos de un DER - Fuente:

<https://bookdown.org/paranedagarcia/database/modelamiento-de-datos.html>

Entidad

Una **entidad** representa un conjunto de objetos reales o conceptuales que tienen existencia dentro del dominio del sistema y que comparten características comunes. Se representan gráficamente mediante un **rectángulo**.

Cada entidad debe cumplir con las siguientes condiciones:

- Puede identificarse de forma única mediante uno o varios atributos.
- Cumple un rol necesario dentro del sistema.
- Se describe por medio de uno o más **atributos**.

Ejemplos de entidades:

En un sistema de gestión de ventas:

- *Cliente.*
- *Producto.*
- *Factura.*
- *Vendedor.*

En un sistema académico:

- *Alumno.*
- *Materia.*
- *Profesor.*
- *Inscripción.*

Tipos de entidades:

Fuertes (o independientes): tienen una clave primaria propia.

Ej.: Cliente, Producto.

Débiles: dependen de otra entidad para su identificación.

Ej.: Detalle de factura, que depende de Factura.

Relación

Una **relación** indica cómo se vinculan dos o más entidades. Se representan mediante un **rombo**, con **líneas** que conectan las entidades involucradas.

Una relación es un conjunto de asociaciones entre instancias de distintas entidades. Reflejan cómo los datos están conectados lógicamente.

Características de una relación:

- Muestra una asociación lógica entre entidades.
- Puede tener atributos propios, si la relación en sí contiene información relevante.
- Puede ser binaria (dos entidades), ternaria (tres entidades) o de mayor grado.
- Tiene una cardinalidad que indica cuántas veces una instancia de una entidad se relaciona con otra.

Ejemplos de relaciones:

- Cliente realiza Pedido.
- Alumno cursa Materia.
- Empleado pertenece a Departamento.
- Proveedor suministra Producto.

Ejemplo con atributos en la relación:

La relación **Empleado** ocupa **Puesto**, con atributo **fecha_inicio** indicando desde cuándo ocupa el puesto.

Tipos de cardinalidad:

1. **Uno a uno (1:1):** Un empleado tiene un solo legajo, y cada legajo corresponde a un único empleado.
2. **Uno a muchos (1:N):** Un cliente puede hacer muchos pedidos, pero cada pedido pertenece a un solo cliente.
3. **Muchos a muchos (N:M):** Un alumno puede cursar varias materias y cada materia puede tener muchos alumnos.

Atributos

Los atributos son las **características** o **propiedades** de una entidad. Se representan con **óvalos** conectados a la entidad correspondiente.

- Cada atributo pertenece a una sola entidad.
- Cuando los atributos adoptan valores específicos, se forma una **instancia** (o registro).

- Para poder distinguir entre distintas instancias, se definen uno o más atributos llamados **identificadores**.

Tipos de atributos:

1. **Simples o atómicos:** no pueden dividirse. Ej.: nombre, precio.
2. **Compuestos:** están formados por subatributos. Ej.: dirección puede tener calle, número, localidad.
3. **Derivados:** se obtienen a partir de otros. Ej.: edad puede calcularse a partir de fecha_nacimiento.
4. **Multivaluados:** pueden tener más de un valor. Ej.: un Alumno puede tener varios teléfonos.

Atributos clave (identificadores):

Un atributo identificador es aquel que permite distinguir de manera única a cada instancia de una entidad. Se lo conoce también como clave primaria (primary key).

Ejemplo:

- En la entidad **Cliente**, el atributo *DNI* podría ser el identificador.
- En la entidad **Artículo**, podría ser *código_producto*.

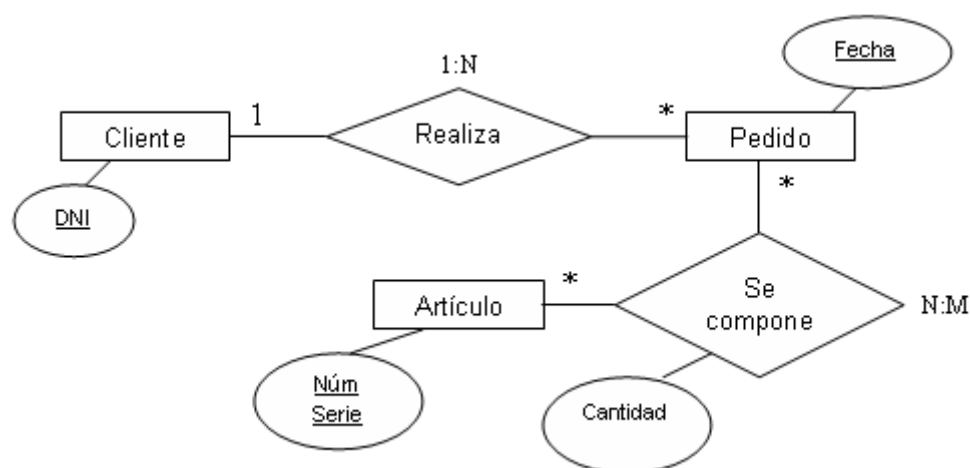


Diagrama Entidad Relación Simple - Fuente:

https://es.wikipedia.org/wiki/Modelo_entidad-relación

Explicación: El atributo "DNI" podría identificar de manera única a una persona dentro de la entidad "**Cliente**".

Balanceo de Modelos

El proceso de balanceo de modelos busca asegurar que las distintas herramientas de modelado utilizadas durante el análisis del sistema estén en coherencia entre sí. Cada herramienta se centra en un aspecto diferente del sistema, por lo que es normal que aparezcan elementos comunes que deben estar alineados.

Hasta ahora vimos distintas herramientas de modelado que se utilizan para analizar un sistema:

- Diagrama de Contexto (DC).
- Diagrama de Flujo de Datos (DFD).
- Diccionario de Datos (DD).
- Diagrama Entidad-Relación (DER).

Cada una de estas herramientas se enfoca en un **aspecto distinto** del sistema y necesitan ser balanceadas.

- El **Diagrama de Contexto** muestra los límites del sistema y su interacción con el entorno.
- El **DFD** representa el flujo de datos y procesos internos.
- El **Diccionario de Datos** define los datos en detalle.
- El **DER** se enfoca en la estructura lógica de los datos.

¿Por qué es importante balancear?

Porque si hay incoherencias entre los distintos modelos, se puede generar confusión, pérdida de información, o errores de interpretación que afecten al desarrollo posterior del sistema. El balanceo garantiza que:

- Los flujos de datos estén definidos en el diccionario.
- Los datos definidos existan en el modelo estructural.
- Los nombres de los elementos sean consistentes.

Debido a que todas estas herramientas representan **visiones distintas de un mismo sistema**, es necesario realizar un proceso de **balanceo**, para verificar que estén en **coherencia entre sí**.



Ilustración: Cada uno de los hombres tiene una visión distinta de las partes del animal.

Errores comunes en el balanceo

1. Definiciones faltantes

Ocurre cuando un elemento aparece en un modelo pero no está definido en otro.

Ejemplo: En el DFD aparece un flujo de datos llamado **Solicitud de Inscripción**, pero este flujo no está detallado en el **diccionario de datos**.

2. Inconsistencia de Nombre o Estructura

Se da cuando un mismo concepto es representado de forma distinta o contradictoria en dos modelos.

Ejemplo: En el DFD se usa el término **Clientes**, mientras que en el diccionario se lo define como **Usuarios**, refiriéndose al mismo grupo. Esto puede causar confusión.

Registro de Balanceo

Una buena práctica es llevar un registro de los elementos a verificar durante el balanceo. Esto puede hacerse con una simple tabla:

Tipo de Problema	Descripción del Caso
Definición faltante	El flujo de "Pago Online" aparece en el DFD pero no en el DD .
Inconsistencia de nombre	En el DER la entidad se llama "Producto", pero en el DFD "Artículo".

Balanceo entre DFD y Diccionario de Datos

Este es uno de los cruces más importantes a revisar:

- Cada **flujo de datos** (cada flecha del DFD) que aparece en un DFD **debe estar definido** en el Diccionario de Datos.
- Cada **almacén de datos** definido en el Diccionario **debe aparecer** representado en algún nivel del **DFD**.
- A su vez, cada proceso que genera o consume datos en el DFD debe estar justificado con los flujos y almacenes definidos.

Este cruce entre modelos asegura que los distintos documentos del análisis del sistema estén alineados y se correspondan correctamente.

El **Diagrama Entidad-Relación** nos permite modelar con precisión la estructura de los datos, reflejando las entidades relevantes del sistema, sus relaciones y atributos. A su vez, el proceso de **balanceo de modelos** asegura que todas las herramientas utilizadas estén alineadas entre sí, lo cual es clave para el desarrollo coherente de un sistema de información.

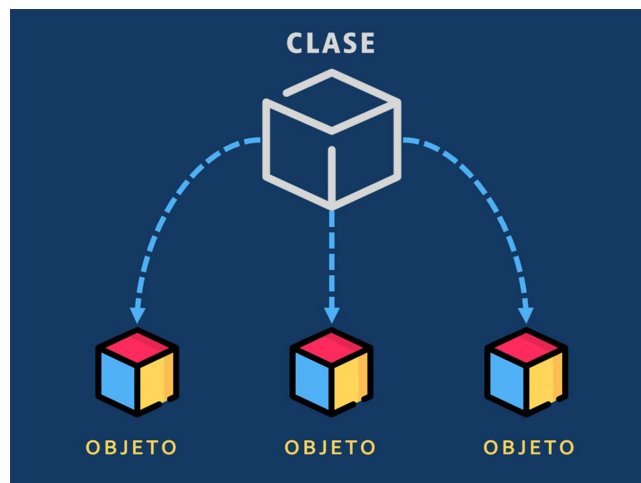
En la próxima clase avanzaremos más sobre los diagramas que se pueden aplicar en el análisis de sistema, tanto en el análisis estructurado cómo en análisis orientado a objetos.

Habiendo mencionado ya la orientación a objetos, ¡Vamos con un poco de programación!



*¡Programación
Orientada a Objetos con Python!*

Programación orientada a objetos



La programación orientada a objetos (**POO**) es un paradigma ampliamente utilizado en el desarrollo de software moderno, y Python ofrece todas las herramientas necesarias para trabajar con este enfoque. Aunque existen algunas diferencias sintácticas y de estilo en comparación con otros lenguajes como Java, la lógica detrás del paradigma sigue siendo la misma: modelar entidades del mundo real como **objetos** que tienen **atributos** (datos) y **métodos** (funciones asociadas).

En esta clase, vamos a ver cómo Python implementa los conceptos fundamentales de la POO: **clases**, **objetos**, **atributos**, **encapsulamiento**, **herencia** y **polimorfismo**.

El uso de **self**: la referencia al objeto actual

En Python, el equivalente a “**this**” en Java u otros lenguajes es la palabra clave “**self**”. Esta palabra se utiliza dentro de una clase para hacer referencia a la instancia actual del objeto.

```
1 class Persona:
2     def __init__(self, nombre, edad):
3         self.nombre = nombre
4         self.edad = edad
5
6     def saludar(self):
7         print(f"Hola, me llamo {self.nombre} y tengo {self.edad} años.")
```

Es decir, cuando querés acceder a un atributo o método del propio objeto dentro de una clase, lo hacés con self.

En este caso:

- **self.nombre** y **self.edad** se refieren a los atributos de la instancia actual.
- El parámetro **self** es obligatorio en todos los métodos de instancia.

Creación de objetos

Una clase define una estructura, pero no tiene valor por sí misma hasta que se crea una instancia de ella, es decir, un objeto.

```
1 class Persona:
2     def __init__(self, nombre, edad):
3         self.nombre = nombre
4         self.edad = edad
5
6     def saludar(self):
7         print(f"Hola, me llamo {self.nombre} y tengo {self.edad} años.")
```

Como siempre decimos, una clase y el objeto no son lo mismo...

```
persona1 = Persona("Ghami", 24)
persona1.saludar()
#Salida: Hola, me llamo Ghami y tengo 24 años
```

Acceso a Atributos

Los atributos definidos dentro de la clase pueden accederse directamente desde la **instancia** del objeto, tal como ocurre en Java.

```
print(persona1.nombre) # Salida: Ghami
persona1.edad = 26
print(persona1.edad) # Salida: 26
```

Encapsulamiento de atributos

El encapsulamiento es un principio de la POO que busca ocultar los detalles internos de un objeto, permitiendo acceder a los atributos solo a través de métodos públicos.

En Python, no existe un verdadero "modificador de acceso privado", pero se pueden simular atributos privados utilizando doble guión bajo (__).

```
class Persona:
    def __init__(self, nombre, edad):
        self.__nombre = nombre
        self.__edad = edad

    def get_nombre(self):
        return self.__nombre

    def set_nombre(self, nuevo_nombre):
        self.__nombre = nuevo_nombre
```

Cómo usarlo:

```
p = Persona("Ana", 30)
print(p.get_nombre()) #Salida: Ana
p.set_nombre("Lucia")
print(p.get_nombre()) # Salida: Lucia
```

Explicación del código:

¿Por qué no son privados?

En Python:

- Atributo privado: `__nombre` -> sólo puede accederse desde dentro de la clase (salvo con trucos).
- Método público: **`get_nombre()`** y **`set_nombre()`** no tienen `__`, por lo tanto no son privados.

Aunque `__nombre` es un atributo privado, vos podés acceder a él de forma controlada usando los métodos públicos `get_nombre()` y `set_nombre()`.

((Se usan `__` para simular privacidad, Python permite el acceso forzado a esos atributos usando `_NombreClase__atributo`, pero no es una práctica recomendada.))

Herencia aplicada

La herencia permite crear clases nuevas que reutilizan y extienden funcionalidades de una clase existente.

```
class Estudiante(Persona):
    def __init__(self, nombre, edad, curso):
        super().__init__(nombre, edad)
        self.curso = curso

    def estudiar(self):
        print(f"{self.nombre} está estudiando el curso {self.curso}.")
```

- **Estudiante** hereda los atributos y métodos de **Persona**.
- **`super()`** se utiliza para llamar al constructor de la clase padre.

En el cuerpo del main se vería algo como esto:

```
estudiante1 = Estudiante("Carlos", 20, "Matemáticas")
estudiante1.saludar()
estudiante1.estudiar()
```

Lo que devolvería sería lo siguiente:

#Hola, me llamo Carlos y tengo 20 años.

#Carlos está estudiando el curso Matemáticas.

Polimorfismo

El polimorfismo permite que distintas clases definan un método con el mismo nombre pero comportamientos **diferentes**. En Python, no es necesario declarar explícitamente una palabra clave como **override**; simplemente se redefine el método en la **subclase**.

```
class Persona:
    def presentarse(self):
        print("Soy una persona.")

class Estudiante(Persona):
    def presentarse(self):
        print("Soy un estudiante.")
```

Ejemplo de cómo usarlo:

```
p = Persona()
e = Estudiante()
p.presentarse() #Salida: Soy una persona.
e.presentarse() # Salida: Soy un estudiante.
```

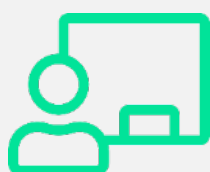
Diferencias con lenguaje de programación java

Característica	Java	Python
Declaración de clase	Con tipos explícitos	Sin tipos explícitos (dinámico)
Atributos privados	Modificadores private, etc.	Convención con “__”
Constructores	public Clase(...)	def __init__(self, ...)
this	Palabra reservada	self como primer parámetro
Polimorfismo / override	Se usa @Override	Se redefine directamente



Hemos llegado así al final de esta clase en la que vimos:

- Diagrama Entidad Relación.
- Uso de diagramas.
- Balanceo entre diagramas.
- Programación Orientada a Objetos en Python.



Te esperamos en la **clase en vivo** de esta semana.
No olvides realizar el **desafío semanal**.

¡Hasta la próxima clase!

Bibliografía

Chen, P. P. (1976). The entity-relationship model—Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9–36.

<https://doi.org/10.1145/320434.320440>

Catedu. (s. f.). Bases de datos relacionales y lenguaje SQL: Diagrama Entidad Relación. Recuperado de

<https://libros.catedu.es/books/bases-de-datos-relacionales-y-lenguaje-sql/page/diagrama-entidad-relacion>

Pressman, R. S., & Maxim, B. R. (2015). *Ingeniería de software: Un enfoque práctico* (8.ª ed.). McGraw-Hill.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.

Vega, M. (s. f.). Casos de uso. Universidad de Granada. Recuperado de <https://lsi2.ugr.es/~mvega/docis/casos%20de%20uso.pdf>

W3Schools. (s. f.). Python programming tutorial. Recuperado de <https://www.w3schools.com/python>

StarUML. (s. f.). Herramienta de modelado UML.

<https://staruml.io/download/>

MGPanel. (s. f.). ¿Qué es la programación orientada a objetos?.

<https://blog.mgpanel.org/post/-que-es-la-programacion-orientada-a-objetos->

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language user guide* (2.ª ed.). Addison-Wesley.

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3.ª ed.). Addison-Wesley.